Walmart Medallion Data Pipeline

MongoDB → PostgreSQL → dbt (Bronze → Silver → Gold)

Incremental PySpark Extraction | 17 dbt Models | 3 Runners | 8 Strict Stages

Nitin — Walmart_DBT

September 2026 | v0.1.0 | github.com/Nitinx12/Walmart_DBT

One pipeline, three runners — **PowerShell**, **Airflow DAGs**, **Docker image** | One warehouse — **walmart_db** with schemas **bronze / silver / gold**

Abstract

This report documents the production-style medallion ETL that mirrors MongoDB operational data into a PostgreSQL warehouse and curates it through `bronze` (raw, 1:1), `silver` (deduped, typed, SCD2) and `gold` (dimensional star/snowflake) layers. The pipeline is implemented once and executed identically by three runners — `pipeline/run_pipeline.ps1` (Windows host), a standalone Docker image (`main.py`), and Airflow DAGs (`airflow/dags/walmart_pipeline_dag.py`, `walmart_health_dag.py`) — over eight strict stages gated by two independent test systems plus Great Expectations. It covers 9 silver and 8 gold dbt models, watermark-based incremental extraction with true upserts, containerisation with two images, CI with a live Postgres integration job, and a Streamlit+Plotly gold dashboard. All commands use `uv` and `uv.lock` is the source of truth.

Keywords — Medallion architecture, PySpark, MongoDB Spark Connector, PostgreSQL 16, dbt Core, Airflow 3.3, Great Expectations, Docker Compose, Streamlit

Contents

1 Introduction & Objectives **4**
- 1.1 Why medallion . . . 4
- 1.2 Project goals . . . 4
- 1.3 Non-goals . . . 4

2 Architecture at a Glance **4**

3 Technology Stack **4**

4 Repository Layout **5**

5 Pipeline — Eight Strict Stages **6**

6 Bronze Ingestion — Watermark Extraction in Detail **6**
- 6.1 Auto-discovery . . . 7
- 6.2 Watermark column priority . . . 7
- 6.3 Incremental pushdown . . . 7
- 6.4 Upsert, not append . . . 7
- 6.5 Nested fields . . . 7
- 6.6 Control tables . . . 7
- 6.7 Validation & reporting . . . 8

7 Silver Layer — Clean, Typed, Historical **8**

8 Gold Layer — Dimensional Star / Snowflake **8**

9 Data Quality — Three Independent Systems **9**

10 Orchestration — One Pipeline, Three Runners **10**

11 Containerisation **10**

12 CI / CD **11**

13 BI Layer — SQL Reports & Dashboard **11**
- 13.1 Hand-written SQL (`sql/`, 25 files) . . . 11
- 13.2 Gold dashboard (`dashboard/`) . . . 11
- 13.3 Existing reports (`reports/`) . . . 11

14 Shared Utilities **12**

15 Running the Pipeline **12**

16 Observability, Security & Conventions **13**

17 Limitations & Roadmap **13**

18 Conclusion **14**

References **14**

A Quick Reference **14**

B Environment Variables (`.env.example`) 14

Introduction & Objectives

Why medallion

Operational MongoDB must stay write-optimised; analytics needs a typed, historical, dimensional warehouse. The medallion pattern isolates those concerns: **bronze** preserves source fidelity, **silver** enforces types and slowly-changing history, **gold** publishes a star schema for BI. Every hand-off between layers is a quality gate — downstream layers only build if the upstream gate passes.

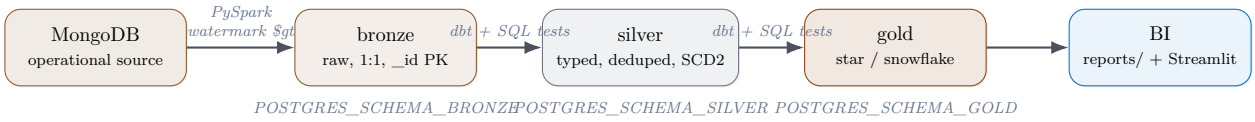
Project goals

1. Replicate *every* user collection from MongoDB to Postgres automatically (no hard-coded list) and keep it incrementally fresh without re-copying unchanged data.
2. Cure source issues once (typing, deduplication, SCD2) so gold consumers never re-implement cleaning.
3. Publish a dimensional star (`fact_order_items` + 7 dims) that powers both SQL reports (`sql/*.sql`) and an interactive dashboard.
4. Gate quality twice at each hand-off: dbt built-ins (`schema.yml`) *and* a standalone SQL suite (`tests/data-quality/` (bronze, silver, gold)), plus a final cross-layer Great Expectations checkpoint.
5. Run the *same* logic three ways — local Windows, Airflow schedule, standalone container — without forking code.
6. Ship like a team would: `uv` lockfile, Ruff + SQLFluff + pre-commit, GitHub Actions CI with a live Postgres integration job, Docker images published to GHCR.

Non-goals

Real-time streaming (this is batch incremental), cross-warehouse federation, and PII masking are out of scope for the current milestone.

Architecture at a Glance



Postgres database is `walmart_db`. Medallion schemas default to `bronze / silver / gold` via `POSTGRES_SCHEMA_*` in `.env` (`.env.example:1` is the template). Control tables `etl_watermarks` and `etl_logs` live alongside bronze collections. All runners set `PYTHONPATH` to the project root so `utils.*` resolves identically.

Technology Stack

Layer	Technology	Version / Notes
Language	Python	3.13 local, 3.12-slim Docker, 3.11 CI
Package mgr	uv	`uv.lock` is source of truth; never hand-edit `pyproject.toml` without `uv lock`

Layer	Technology	Version / Notes
Compute	PySpark	3.5.5 (pinned) — required by `mongo-spark-connector_2.12:10.4.0`; 4.x is binary-incompatible (SPARK-4 Catalyst change)
Source	MongoDB	Operational DB, auto-discovered collections
Warehouse	PostgreSQL	16 — `walmart_db`, JDBC via `jars/postgresql.jar`
Transform	dbt Core + dbt-postgres	1.12.0 / 1.11.0 — 17 models, 100+ tests
Orchestration	Apache Airflow	3.3.0, LocalExecutor (apiserver, scheduler, metadata DB; tasks run in the scheduler container)
Quality	Great Expectations	1.21.0 — suites per layer in `gx/suites/`, run via `python -m gx.run`
BI	Streamlit 1.62 + Plotly 7.0	Live gold dashboard at `dashboard/app.py`
Lint / Format	Ruff 0.16 + SQLFluff 4.2 + pre-commit	`ruff check / format`, `sqlfluff lint -dialect postgres`
Containers	Docker / Compose	`docker/Dockerfile.pipeline` (standalone) + `docker/Dockerfile.airflow`
CI	GitHub Actions	lint → smoke → unit → DAG → Docker → integration (live Postgres, 7 jobs)

JDK is 17 (`docker/Dockerfile.pipeline` and dev container) — PySpark 3.5's Mongo connector is untested on JDK 21, so the pin is intentional (`AGENTS.md` convention).

Repository Layout

Listing 1: Top-level layout (trimmed). Mirrors `AGENTS.md`.

```
airflow/        dags/{walmart_pipeline_dag.py (mwf_morning, rest_night),
   walmart_health_dag.py} + logs/plugins (bind-mounted)
docker/         Dockerfile.pipeline, Dockerfile.airflow, compose.yml,
   entrypoint.sh
docs/           architecture.md, airflow.md, dbt.md, docker.md, pipeline.
   md, scripts.md, ... (14 files)
jars/           mongo-spark-connector + postgresql.jar (offline Spark
   startup)
pipeline/       run_pipeline.ps1 (+ .bat shim)
scripts/        flat run_*.py wrappers (run_tests, run_gx,
   run_health_check, seed_demo_db, ...) + helpers
src/jobs/       extract.py, sql_test.py, ci_seed_bronze.py (pipeline
   runtime)
src/utils/      engine.py, connection.py, logger.py (shared infra)
sql/            analytics/, functions/, triggers/, ... (25 hand-written
   files, organised by area)
tests/          unit/ (28 tests) + data-quality/{bronze (3), silver (9),
   gold (7)} -- standalone SQL suite
gx/             checkpoints, expectations, suites, validation_definitions
dbt/            dbt_project.yml, models/{silver (9), gold (8)}, seeds,
   snapshots, macros
dashboard/      home.py, pages/, lib/, helpers/ (multipage Streamlit +
   Plotly app)
reports/        brand_report.md, category_report.md, report.tex (this file
   ) -> report.pdf
```

```
main.py          standalone Docker entry point (mirrors ps1 stage-for-stage
    )
```

Coding conventions: `snake_case`, type hints on every signature, one function = one responsibility (see `src/jobs/extract.py` stage split), short `// why not what` comments, `uv run ruff format/check` before commit.

Pipeline — Eight Strict Stages

The pipeline has a fixed order. If stage order or the definition of "success" changes, `pipeline/run_pipeline.ps1` *and* `airflow/dags/walmart_pipeline_dag.py` must change together (convention, not enforced — `AGENTS.md: Pipeline Stage Rule`).

#	Stage	Runner command	Gate
0	Preflight	`uv run python -c "import pyspark; print(...)"` + prefix check `3.5.x`	Version mismatch → fail fast (ps1) / install guard (container refuses to self-heal)
1	Extract	`uv run python src/jobs/extract.py`	Per-collection `VALIDATION FAILED` → non-zero exit
2	Bronze SQL tests	`uv run python src/jobs/sql_test.py tests/data-quality/bronze`	3 checks; any violating rows → FAIL
3	dbt silver	`uv run dbt run -select silver` + `uv run dbt test -select silver`	`schema.yml` tests; `dbt test` failure stops stage
4	Silver SQL tests	`uv run python src/jobs/sql_test.py tests/data-quality/silver`	9 checks
5	dbt gold	`uv run dbt run -select gold` + `test`	8 models, 7 dims + 1 fact
6	Gold SQL tests	`uv run python src/jobs/sql_test.py tests/data-quality/gold`	7 checks
7	Great Expectations	`uv run python -m gx.run -layer all`	Bronze/Silver/Gold suites; exit 1 = data failure, 2 = build error

Stop-on-first-failure: `run_pipeline.ps1` (`Stop-Pipeline`) and `main.py` (`raise`) halt immediately; the Airflow DAGs use the default `all_success` trigger to do the same — downstream tasks go to `upstream_failed` with no extra guard.

PowerShell, Python, and Airflow share one mental model: the header "`STEP n / 7`" (`main.py` `TOTAL_STEPS = 7` plus stage 0 = 8 stages) stays honest via an assert. Logs go to `logs/pipeline_YYYY-MM-DD.log` (`src/utils/logger.py`) plus `{schema}.etl_logs` audit rows.

Bronze Ingestion — Watermark Extraction in Detail

`src/jobs/extract.py` is the only place that touches MongoDB.

Auto-discovery

`discover_collections()` lists `mongo_db.list_collection_names()` and drops any name starting with `system.`. No collection list is ever hard-coded, and `-tables` is an optional filter for ad-hoc runs.

Watermark column priority

Per collection, a single document sample is inspected and the first present candidate wins:

`updated_timestamp` ≻ `updated_at` ≻ `created_timestamp` ≻ `created_at` (watermark priority in `src/jobs/extract.py`)

`updated_*` is preferred because documents mutated after insert (e.g. order status → `Cancelled`) must be re-synced, not just newly-inserted rows. If none is present, the collection falls back to full reload every run with a warning.

Incremental pushdown

When a watermark column and a prior `last_watermark_value` exist, the Spark read is filtered server-side:

Listing 2: Pushdown filter — string comparison is intentional

```
watermark_str = since.strftime("%Y-%m-%d %H:%M:%S")
pipeline = json.dumps([{"$match": {col: {"$gt": watermark_str}}}])
reader.option("aggregation.pipeline", pipeline)  # src/jobs/extract.py
```

The comparison is string-to-string because the source stores these timestamps as `"YYYY-MM-DD HH:MM:SS"` strings, not BSON dates — MongoDB never matches across BSON types in `$gt`.

Upsert, not append

`_id` (cast to string, `sanitize_for_postgres`) is the merge key. Existing tables get a scratch staging table `_stg_{collection}` written with `mode("overwrite")`, then a single

```
WITH upsert AS (
  INSERT INTO "bronze"."orders" ("_id", ...)
  SELECT ... FROM "bronze"."_stg_orders"
  ON CONFLICT ("_id") DO UPDATE SET ...
  RETURNING (xmax = 0) AS inserted
)
SELECT count(*) FILTER (WHERE inserted) AS rows_inserted,
       count(*) FILTER (WHERE NOT inserted) AS rows_updated
FROM upsert;  -- src/jobs/extract.py
```

Listing 3: Merge — inserted vs updated via xmin

A unique index on `_id` is ensured first (`ensure_unique_id_index`); if creation fails (duplicate _id from a prior append-only run), the run falls back to plain `append` and warns — never silently corrupts.

Nested fields

`StructType / ArrayType / MapType` columns are serialised to JSON strings via `to_json()` — the only "lossy" step, required because plain JDBC has no native struct type. Flattened field names are reported in the Rich final panel.

Control tables

Both live in the bronze schema:

- `{bronze}.etl_watermarks` — one row per collection: `incremental_column,last_watermark_value,last_run_mode,rows_inserted/updated,updated_at`. Upserted via `ON CONFLICT (table_name)`.
- `{bronze}.etl_logs` — one audit row per collection per run: mode, counts before/after, validation status, `error_full` traceback, duration. Best-effort, never fails the run.

`ensure_control_tables` is idempotent and forward-migrates columns added in later versions.

Validation & reporting

After each merge, `validate_collection` recounts the target and expects `before + inserted`. A Rich report prints per-collection rows, watermark state, flattened fields, and failures (`render_report`); the same data lands in both the daily log file and `etl_logs`.

CLI: `-tables`, `-full-refresh` (truncates before load), `-dry-run`, `-watermark-column` override.

Silver Layer — Clean, Typed, Historical

`dbt/dbt_project.yml` materialises all silver models as `table` in schema `silver`. Nine models mirror bronze tables 1:1 but add:

Model	Key cleaning	Notable logic
`silver.brands`	dedup on `_id`, trim/case normalisation	Coalesce brand name variants
`silver.categories`	dedup, null handling	Canonical category list
`silver.customers`	dedup, email lower, phone normalise	`is_active` flag
`silver.employees`	dedup, date parsing	Joins to stores
`silver.orders`	timestamp parsing, status canonicalisation	`order_timestamp` typed
`silver.order_items`	`line_amount = qty * unit_price` recomputed	FK to orders + products
`silver.payment_methods`	dedup, type normalisation	Lookup for gold dim
`silver.products`	dedup, price typing, FK checks	Links brands + categories
`silver.stores`	dedup, geo normalisation	Store master

Each model has a `schema.yml` with `not_null`, `unique`, `relationships` and `accepted_values` tests. The silver grain is "one row per source document" — no aggregation yet.

`tests/data-quality/silver/` contributes 9 additional PL/pgSQL checks (row counts, orphan FKs, distinctness) executed by `src/jobs/sql_test.py`.

Gold Layer — Dimensional Star / Snowflake

`dbt/dbt_project.yml` materialises gold as `table` in schema `gold`. Eight models:

Model	Type	Grain
`gold.dim_stores`	Dimension	One row per store

Model	Type	Grain
gold.dim_customers	Dimension	One row per customer
gold.dim_products	Dimension (snowflake via brands/categories)	One row per product
gold.dim_brands	Sub-dimension of products	One row per brand
gold.dim_categories	Sub-dimension of products	One row per category
gold.dim_orders	Dimension	One row per order header
gold.dim_payment_methods	Dimension	One row per payment method
gold.fact_order_items	Fact (incremental merge)	One row per order line; FKs to dim_orders, dim_products

Key design points:

- fact_order_items.sql is the only incremental gold model — unique_key=order_item_id, incremental_strategy='merge', 3-day lookback window (updated_timestamp>=MAX(updated_timestamp)-3days) to catch late-arriving updates.
- Dimensions are full-refresh tables rebuilt from silver each run — cheap at this scale and avoids SCD bookkeeping where not needed.
- gold_loaded_at = CURRENT_TIMESTAMP stamps lineage.
- The star is snowflaked only at products → brands and categories — intentional. Brand/category reports in reports/ read straight from those sub-dims.

Gold is gated by dbt test -select gold plus 7 standalone checks under tests/data-quality/gold/.

Data Quality — Three Independent Systems

System	Location	Runner	Semantics
dbt tests	dbt/models/*/schema.yml	dbttest--selectsilver|gold	Column/table contracts (null, unique, FK, accepted values)
SQL suite	tests/data-quality/*/*.sql	src/jobs/sql_test.py	SELECT returning violating rows = FAIL; DO$$RAISEEXCEPTION = FAIL. One connection per layer, rollback on fail
Great Expectations	gx/suites/	python -m gx.run -layer all	One checkpoint per layer ({layer}_checkpoint), Fluent API 1.x; exit 1 = data failure, 2 = build error

The SQL suite auto-detects mode per file via `psycopg2 ResourceClosedError` (tests/unit/test_sql_test.py). GX checkpoints reconcile on re-run instead of duplicating, and all requested layers run even if an earlier one fails unless `-fail-fast` is passed (gx/run.py).

Together: bronze has 3 SQL checks, silver 9, gold 7, plus ≈23 GX expectation suites and 100+ dbt tests.

Orchestration — One Pipeline, Three Runners

Runner	Entry point	How it loads env & runs
Windows host	`pipeline/run_pipeline.ps1` + `run_pipeline.bat` shim	`Import-DotEnv` overwrites process env; `Assert-Prerequisites` checks paths; `Invoke-PipelineCommand` shells out via `uv`
Standalone container	`main.py` (used by `docker/Dockerfile.pipeline` `ENTRYPOINT`)	`load_dotenv` via `setdefault` (container env wins); `in_container()` check refuses to self-heal PySpark inside a built image
Airflow DAGs	airflow/dags/walmart_pipeline_dag.py (walmart_mwf_morning, walmart_rest_night) + walmart_health_dag.py (walmart_hourly_health_check)	Every `@task.bash` prefixes `_PREAMBLE`: parse `.env`, rewrite `localhost` to `host.docker.internal`, force PYSPARK_PYTHON=/app/.venv/bin/python, set DBT_PROFILES_DIR=/app/dbt

The pipeline DAGs form a linear chain preflight->extract->bronze_sql_tests->dbt_silver_run->dbt_silver_test->silver_sql_tests->dbt_gold_run->dbt_gold_test->gold_sql_tests->great_expectations_tests->collect_results->choose_email->email_success/failure with `retries=1, max_active_runs=1`. Fail-fast is free from Airflow's `all_success`.

"Localhost problem" (`docs/architecture.md`): `.env` keeps `localhost` for local `uv run`; only consuming layers rewrite — the DAG `_PREAMBLE` and `docker run -env-file` overrides. `.env` itself is never edited.

Containerisation

Two images, two jobs (`docs/docker.md`):

- `docker/Dockerfile.pipeline` — standalone runner: `python:3.12-slim + JDK 17 + uv`, copies `jars/`, installs from `uv.lock`, sets `RUNNING_IN_DOCKER=1`, `ENTRYPOINT ["python","main.py"]`.
- `docker/Dockerfile.airflow` — scheduler base: `apache/airflow:3.3.0 + JDK 17 + uv`, same `jars/` layer, used by `docker/compose.yml` for 4 services (metadata, apiserver, migrate, scheduler) on `LocalExecutor`.

`docker/compose.yml` mounts the project root to `/app` and shadows `/app/.venv` with an anonymous volume so the container's Linux venv never clobbers the host's Windows venv. The shared

Airflow block sets `extra_hosts: host.docker.internal:host-gateway` so tasks reach host Postgres/Mongo, and pins `AIRFLOW__CORE__EXECUTION_API_SERVER_URL` plus shared API/JWT secrets so scheduler-spawned tasks authenticate to the Execution API.

Dev container (`.devcontainer/devcontainer.json`) mirrors the standalone Dockerfile (JDK 17, uv preinstalled) so PySpark works without local Java.

CI / CD

`.github/workflows/ci.yml` (7 jobs, `PYTHON_VERSION 3.11`, `AIRFLOW_VERSION 3.3.0`):

1. **lint** — `ruff check`, `ruff format -check`, `sqlfluff lint dbt/models sql/ tests/ -dialect postgres`
2. **dashboard-smoke** — installs *only* dashboard deps (no `databricks-sql-connector`), asserts `databricks` not importable, then imports with dummy env — catches the unconditional-databricks regression that broke Streamlit Cloud.
3. **unit-tests** — `pytest tests/unit -v -cov` (28 tests, mocked, no live DB)
4. **smoke-tests** — fast import/startup checks without live services
5. **dag-integrity** — installs Airflow via constraints file and asserts `DagBag` loads `airflow/dags` with no import errors or cycles.
6. **docker-build** — builds both `docker/Dockerfile.pipeline` and `docker/Dockerfile.airflow` with Buildx cache, `lfs: true` so `jars/*.jar` are real files.
7. **integration** — live Postgres 16 service; `ci_seed_bronze.py` seeds bronze from fixtures, then the exact gate sequence: bronze SQL → `dbt run/test silver` → silver SQL → `dbt run/test gold` → gold SQL → GX `-layer all`. Uses absolute `DBT_PROFILES_DIR` and `PYTHONPATH` fixes documented inline.

Images are published to GHCR on merge/tag; releases are cut via `CHANGELOG.md`.

BI Layer — SQL Reports & Dashboard

Hand-written SQL (`sql/`, 25 files)

`sql/` is organised by area — `analytics/` (brand/category revenue share, payment mix, ranking, Pareto, market basket, ABC, cohort), `functions/` (PL/pgSQL reporting functions), `triggers/`, plus `extensions/`, `metadata/`, `procedure/` and `scripts/`. They query gold directly and are the ancestors of `reports/brand_report.md` and `category_report.md`.

Gold dashboard (`dashboard/`)

`dashboard/` is a multipage Streamlit app with Plotly charts: `home.py` entry point, `pages/` per area (Sales, Products, Customers, Categories), SQL in `lib/queries.py` (parameterised, aggregated in Postgres, cached), styling in `helpers/theme.py` and shared cards in `helpers/components.py`. Filters build an immutable key; KPIs show period-over-period deltas; graceful empty states when gold is unpopulated. Runs with `uv run streamlit run dashboard/home.py` or on Streamlit Cloud with `st.secrets` fallback.

Existing reports (`reports/`)

`reports/brand_report.md` (7 brands, $18.9M total, Apple 15.75% leader) and `reports/category_report.md` (Electronics 17.76% leader, Home 14.34% outlier) are the reference BI outputs — both

derived from gold CTEs that `LEFT JOIN` `dim_*->fact_order_items` so zero-sales entities remain visible.

Shared Utilities

`src/utils/engine.py`
Loads `.env` at import, validates required vars (`POSTGRES_*`, `MONGO_*`) with `OSError` on missing, warns on optional `POSTGRES_SCHEMA_*` and Databricks vars, casts `POSTGRES_PORT` to int. Hard-fails fast so every script gets a clear error.

`src/utils/connection.py`
Cached factories: `get_postgres_engine()` (SQLAlchemy, SSL params), `get_mongo_db()` (pymongo with ping), `get_databricks_connection()` (lazy `databricks_sql_connector`, `ModuleNotFoundError` if extra not installed — dashboard-safe).

`src/utils/logger.py`
`get_logger(name)` returns a logger with console + daily file handlers (`logs/pipeline_YYYY-MM-DD.log`), `propagate=False`, configurable levels. Used by extract, dashboard, and GX runner.

Patching for tests is via `tests/unit/test_*.py` with `unittest.mock` and small fixtures — never real DB data.

Running the Pipeline

Listing 4: Local (Windows) — mirrors README "Run it".

```
# 1. Environment
uv venv
.venv\Scripts\activate          # Windows; source .venv/bin/activate on
    Linux/Mac
uv sync                         # from uv.lock
cp .env.example .env            # fill POSTGRES_*, MONGO_*, optional
    DATABRICKS_*

# 2. One-command pipeline (8 stages)
./pipeline/run_pipeline.ps1
# or: uv run python main.py
# or: uv run python src/jobs/extract.py --dry-run --tables orders,
    customers

# 3. Individual gates
uv run python src/jobs/sql_test.py tests/data-quality/bronze
uv run dbt run --select silver && uv run dbt test --select silver
uv run python -m gx.run --layer all

# 4. Dashboard
uv run streamlit run dashboard/home.py

# 5. Quality (pre-commit)
uv run ruff format . && uv run ruff check . && uv run sqlfluff lint dbt/
    models --dialect postgres
uv run pre-commit run --all-files
uv run pytest tests/unit -v
```

Listing 5: Docker & Airflow

```
# Standalone container
```

```
docker build -f docker/Dockerfile.pipeline -t walmart-pipeline .
docker run --env-file .env walmart-pipeline

# Full Airflow stack (LocalExecutor)
docker compose -f docker/compose.yml up -d
# UI at http://localhost:8080, DAGs: walmart_mwf_morning,
    walmart_rest_night, walmart_hourly_health_check
```

Health and security scripts: `uv run python` scripts/run_health_check.py and scripts/run_security_check.py (see `docs/health.md`).

Observability, Security & Conventions

- Git workflow: one file/logical change per commit, `git add <file>` + area prefix (`extract:` / `dbt:` / `airflow:` / `docker:` / `tests:` / `src:` / `docs:`) (`AGENTS.md`, `docs/git-workflow.md`), rebase before push, no `.venv`, `__pycache__`, `.env`, or anything under `data/raw` or `data/processed` committed.
- Secrets: `.env` is git-ignored; `.env.example` is the template; `detect-private-key` pre-commit hook; Databricks token never baked into images.
- Logging: Rich console + structured file logs + `etl_logs` table — three places to diagnose a failed collection without re-running.
- Idempotency: control tables and watermark upserts are `CREATE IF NOT EXISTS / ON CONFLICT` — re-running never duplicates state.
- Offline Spark: `jars/postgresql.jar` is vendored; the 5 Mongo connector jars are vendored when present in `jars/` — otherwise resolved once via Maven and a tip is printed (`extract.py:304`).

Limitations & Roadmap

Area	Next step
Watermark type	Source timestamps are strings; migrating Mongo to BSON dates would allow native date pushdown and remove the string-format coupling.
SCD2	Silver history is dedup-only today; true SCD2 snapshots (dbt snapshots on `dim_customers / dim_stores`) are sketched in `docs/dbt.md` but not yet materialised.
Gold incremental	Only `fact_order_items` is incremental; dimensions are full refresh — fine at current scale, revisit if product/customer counts grow 10×.
Streaming	Batch-only; CDC via Debezium or Mongo change streams would enable near-real-time bronze.
PII / masking	No column-level masking in silver; add views or dbt post-hooks for sensitive fields before exposing gold externally.
Data Docs	GX checkpoints have no `UpdateDataDocsAction` yet — add once `gx/uncommitted` is published.
Tests	Bronze `schema.yml` tests are lighter than silver/gold — expand with `dbt_expectations` for cross-column rules.

Conclusion

The Walmart medallion pipeline delivers a clean separation between operational and analytical concerns without duplicating logic across execution environments. Auto-discovery plus watermark upserts keep bronze fresh and auditable; dbt turns that bronze into a typed silver and a dimensional gold that is directly queryable for reports and a live dashboard; two independent test systems plus Great Expectations gate every hand-off; and a single stage definition runs identically on a developer laptop, in Docker, and on a schedule in Airflow. The codebase is small enough to read in an afternoon (`extract.py` + 17 models + `sql_test.py` + `run.py`) yet strict enough to catch schema drift, load errors, and data regressions before they reach gold.

References

1. Project repository — github.com/Nitinx12/Walmart_DBT
2. Live dashboard — Streamlit Cloud
3. dbt documentation — docs.getdbt.com
4. Great Expectations 1.x Fluent API — docs.greatexpectations.io
5. MongoDB Spark Connector — mongodb.com/docs/spark-connector
6. Apache Airflow 3.3 — airflow.apache.org

Quick Reference

Task	Command
Create env	`uv venv`
Install deps	`uv sync`
Add dep	`uv add <pkg>`
Run extract	`uv run python src/jobs/extract.py`
Run dbt tests	`uv run dbt test -select silver`
Run SQL suite	`uv run python src/jobs/sql_test.py tests/data-quality/<layer>`
Run GX	`uv run python -m gx.run -layer all`
Format	`uv run ruff format .`
Lint	`uv run ruff check .`
Stage one file	`git add <file>`
Commit	`git commit -m "<area>:  <summary>"`

Environment Variables (`.env.example`)

```
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DATABASE=walmart_db
POSTGRES_USERNAME=postgres
POSTGRES_PASSWORD=changeme
POSTGRES_SCHEMA_BRONZE=bronze
POSTGRES_SCHEMA_SILVER=silver
POSTGRES_SCHEMA_GOLD=gold
```

```
MONGO_URI=mongodb://localhost:27017
MONGO_DB=walmart
PYSPARK_PYTHON=python
PYSPARK_DRIVER_PYTHON=python
DATABRICKS_HOST=
DATABRICKS_HTTP_PATH=
DATABRICKS_TOKEN=
DATABRICKS_CATALOG=walmart
DATABRICKS_SCHEMA=public
```